

Master's thesis

Master's Programme in Computer Science

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

August 20, 2026

Faculty of Science

University of Helsinki

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki,Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Study programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ohjaajat — Handledare — Supervisors

Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages
Master's thesis	August 20, 2026	28 pages

Tiivistelmä — Referat — Abstract

Abstract to be written last.

ACM Computing Classification System (CCS)
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords

Rust, C++, RAN control plane, tail latency, memory safety, fault injection, incremental adoption

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — övriga uppgifter — Additional information

Software study track

Contents

1 Introduction 1

2 Background: Systems Languages for Latency-Critical Software 3

3 Problem Context: UE Context Management and Predictability 5

4 Objectives 7

5 Research Methodology 9

6 Design and Implementation 11

7 Experimental Setup and Evaluation 13

8 Discussion 17

9 Conclusion and Future Work 21

Statement on the Use of AI Tools 23

Bibliography 25

1 Introduction

- The 5G RAN control plane (the gNB-CU-CP) now runs as software on ordinary servers. It is large, it keeps state for every connected device, it serves many requests at once, and the 5G target for control plane latency, the transition from a battery-efficient state to continuous data transfer, is 10 ms [1]. One defect, or one regression in the slowest requests, therefore degrades the whole site. The work is carried out at Nokia on a UE Context Management service modelled on a real gNB-CU-CP component.

- This software is written mostly in C++. C++ gives the control over memory and timing that such a system needs, but it leaves memory-safety and concurrency mistakes to the programmer [2]. Rust promises the same performance while rejecting those mistakes at compile time [3]. The choice has a long horizon: 3GPP has allocated Release 20 to the 6G studies and Release 21 to the normative work, with migration from 5G in scope [4], so a language chosen for the control plane of the current generation is inherited by the next.

- The problem is a lack of evidence. Telecom vendors are under growing pressure to move to memory-safe languages [5], and the closest prior work is a cluster of at least five Ericsson master's theses, none of which answers the question as posed here. Three keep the runtime in play alongside the language, comparing Boost.Fibers, goroutines, Tokio, Monoio, Glommio and DPDK Eventdev rather than the languages that host them [6, 7, 8]. The other two report execution time and peak memory rather than tail behaviour under rising load [9, 10]. Emberg and Wadsten come closest in method, with a matched C++ and Rust pair joined by a C wrapper, but measure a stateless transformation [10]. No study holds the specification, concurrency model, and build settings fixed for a stateful, concurrent subsystem while sweeping past saturation and measuring incremental migration cost.

- That evidence is also concentrated in one vendor and untested outside it, while on the other side there is no study of Rust among Nokia's indexed publications at all. The nearest peer-reviewed position from that side argues for making

memory-error sanitisation practical rather than for replacing it with a compile-time guarantee [11], which is the strategy this thesis tests against.

- Four research questions frame the work. RQ1 compares latency predictability (median, p99, p99.9, maximum) under increasing load and overload with the concurrency model held fixed [12]. RQ2 asks which defect classes Rust removes at compile time, at what stage the rest are detected, and which classes it does not prevent. RQ3 asks what overhead Rust's safety model imposes on throughput, CPU, and memory. RQ4 asks what incremental adoption costs in effort, FFI interoperability, and risk inside an existing C++ RAN codebase.

- The work contributes five things: a reproducible, telecom-relevant benchmarking framework; a frozen shared specification with two parity-tested implementations; a quantitative comparison that emphasises tail latency under overload; a defect-class taxonomy that marks where Rust's compile-time guarantee stops; and evidence-based guidelines for incremental adoption, where the dominant cost is the learning curve rather than run-time [13].

2 Background: Systems Languages for Latency-Critical Software

- In the NG-RAN architecture the base station (gNB) is split into a central and a distributed unit, and the central unit is split again into a control plane and a user plane. UE Context Management sits in the control plane, the gNB-CU-CP, and reaches its neighbours over the standardised F1, E1, and NG interfaces [14].
- Deadlines are met or missed because of predictability, not average speed. The metrics that matter are therefore the p99 and p99.9 latencies and the jitter around them [12]. They have to be measured past the saturation point and recorded without coordinated omission, that is, without letting a load generator that has fallen behind hide its own slowest requests [15, 16].
- C++ offers abstractions that cost nothing at run time, deterministic cleanup through RAII, and no garbage collector. Safety, however, remains the programmer's responsibility [2], and sanitizers report only the errors that a given test run happens to reach.
- Rust has the same run-time model and also no garbage collector, but it checks memory safety and freedom from data races at compile time, using ownership, borrowing, and thread-safety traits such as `Send` [3, 17]. Code that needs to step outside those rules must be marked `unsafe`, which keeps such code small and easy to find.
- The comparison is therefore C++ against Rust. Garbage-collected languages are left out because their pauses are incompatible with the latency targets. The concurrency model is held fixed in both implementations, and the C ABI is treated as the realistic adoption path. No earlier study isolates the language this way for a stateful, concurrent subsystem [6, 10].

3 Problem Context: UE Context Management and Predictability

- UE Context Management owns the lifecycle of one context per connected device. Each context moves through four states, Idle, Connected, Reconfiguring, and Released, driven by a small set of lifecycle operations. An operation that is illegal in the current state returns a typed error instead of being carried out.
- The subsystem was chosen because it is both representative and small enough to keep under control. It has mutable state per device, a strict lifecycle, heavy concurrency, and hard latency targets [1, 18], and it is still simple enough to specify precisely and to test for identical behaviour in two languages.
- Five defect classes fall inside Rust's guarantee, and each has a natural form in this subsystem: use-after-free, double-free, iterator invalidation, data race, and unguarded shared mutation. What the guarantee rests on is aliasing control and data-race freedom [17]. The catalogue is this study's own.
- Five further classes are included on purpose from outside that guarantee: a lock-order deadlock, a lost update, a leak through a reference cycle, an integer overflow in a release build, and the blast radius of a failed shard worker. They keep the study from confirming a foregone conclusion and show instead where the guarantee stops. Two of them return the opposite of the expected result, with the C++ tooling reporting what Rust's does not.
- Scale is what makes the tail matter. One procedure triggers many operations and one site serves many devices, so a p99.9 measured on a single operation becomes a steady rate of degraded procedures in the field [12]. This is why the load sweep is pushed past the point where the service starts to fall behind.

4 Objectives

- A documented literature search of the citation databases and of the Nordic thesis repositories, where the closest work turns out to live, found no study that holds a stateful control-plane subsystem fixed while measuring tail latency of Rust and C++ past saturation under the same concurrency model [6, 8, 10].
- Five objectives follow from the research questions: measure predictability (O1), establish which defect classes are prevented (O2), measure the resource overhead (O3), build and measure a migration prototype (O4), and publish reusable artifacts and guidance (O5).
- Three hypotheses are stated in advance, so that the measurements can refute them. H1: below saturation the two implementations perform the same. The prior evidence points to near-parity with a measurable residual rather than to equality, a Rust network driver running 4% slower than its C reference while executing 63% more instructions per packet [19]. H2: Rust prevents at compile time the defect classes for which C++ can only offer best-effort detection. H3: the cost of calling Rust from C++ is negligible.
- The work succeeds if the evidence is good enough to base a decision on: repeated runs reported with confidence intervals [20], defect claims shown on programs that really compile and run, migration cost read off a working prototype, and every number reproducible from the published artifacts.
- The scope is drawn narrowly to protect internal validity. The full gNB, the PHY and L2 layers, the user plane, captured traffic traces, managed languages, and production operations are all left out.

5 Research Methodology

- The approach is empirical and constructive. Instead of comparing two systems that already exist and differ in many ways, the study builds a matched pair of implementations from one frozen specification, so that the language is the only systematic difference between them. Both use an identical thread-per-shard concurrency model, because that structuring itself changes tail latency [21].

- RQ1 and RQ3 are answered with an open-loop load sweep that runs from light load into overload on pinned cores. Open-loop means the generator keeps to its send schedule instead of waiting for replies. Latency is recorded in identically configured HDR histograms [16], and resource use is sampled through the operating system. The results so far come from a dedicated dual-socket Xeon host running as a KVM guest. That host resolves the overload divergence and the resource comparison, but not the far tail below saturation: the frequency governor is not observable from inside the guest and CPU steal is non-zero. Locking frequency requires the bare-metal campaign.

- RQ2 is answered by controlled fault injection [22]. A fixed catalogue of defects is written in both languages, and the expected outcome of each one is recorded in advance, so that a surprise shows up as a discrepancy instead of being explained away afterwards.

- RQ4 is answered by a working prototype in which C++ drives the Rust core through a foreign-function interface, the seam being where cross-language memory-management defects concentrate [23]. It yields three measurements: how large the seam is, how much `unsafe` code it needs, and what one call across it costs.

- Fairness is the central experimental control: the specification is frozen, allocator and optimisation settings are matched, and every point where the two languages had to differ is written down in a forced-idiom log. Reporting follows the guidance that a single figure invites conclusions run-to-run variation does not support [20]: medians with nonparametric bootstrap confidence intervals [24], and exact paired

permutation tests corrected for multiple comparisons by the Holm procedure [25]. All data is synthetic, so no data-protection concerns arise.

6 Design and Implementation

- Nothing is implemented until the specification is frozen. It fixes the public API, the UE state machine, the data structures and their algorithmic strategy, the locking and sharding granularity, and the concurrency model. Both implementations conform to it, so what the comparison measures is the language and not a difference of design or of skill. The specification is itself a deliverable and the study's primary control against implementation bias.

- One concurrency model is used on both sides, shared-nothing and thread-per-shard, because that structuring is known to change tail latency by itself [21]. The identifier space is partitioned into shards, each owned by a single worker thread that alone reads or mutates it, so no lock guards context state. Callers submit a request to the owning shard's bounded queue and wait on a single-use completion handle. The queue blocks when it is full, which is deliberate: a drop policy would hide overload, and this study needs overload to appear as latency. The only shared mutable state either version permits is one atomic counter, and no control flow depends on its value.

- Fairness is enforced below the design as well as in it. Both builds link the same `jemalloc` and keep a system-allocator build as a control, both compile at matched optimisation, and both record latency through one HDR histogram algorithm configured identically [16]. Hashing is pinned on both sides, SplitMix64 for shard assignment and FNV-1a inside the map, because C++'s default hash of an integer is typically the identity function while Rust's is cryptographically strong. Left as they came, the two standard libraries would have been the measurement.

- Where the languages cannot satisfy a requirement the same way, the divergence is written into a forced-idiom log and then measured. Rust takes its bounded blocking queue from `sync_channel`, while C++ through C++23 has no such primitive and assembles one from a mutex and two condition variables. C++ has no payload-carrying enumeration in this idiom either, so a request becomes a tagged struct holding every argument field any operation might need, 160 bytes against Rust's 96, answered through a `variant` the caller has to unwrap. The

net of all of it is a Rust core 18% longer in code lines whose total cyclomatic complexity is 15% lower [26], the branchiest function in either core being the C++ worker loop that dispatches that `variant`. The Rust library contains no `unsafe` at all.

- Parity is tested and not assumed. Language-neutral vectors run in both implementations and their transcripts are diffed operation by operation. Because a cross-diff cannot tell “both correct” from “both wrong in the same way”, a third implementation in Python acts as an independent oracle: it has to reproduce all 223 hand-derived expectations before it may generate any of its own, and a campaign of roughly 793 000 generated operations found no divergence. The transition matrix is enumerated exhaustively instead of sampled, the C++ side runs under AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer and the Rust side under Miri, and line coverage is reported, so a clean sanitiser result is bounded by the code that actually ran.

7 Experimental Setup and Evaluation

- The workload is synthetic and spec-justified. A deterministic driver walks each UE through the connection lifecycle the 3GPP specifications define, and because its draw order is pinned, the per-operation-kind counts come out identical in both languages for a given seed, which doubles as a parity signal. Each sweep point is repeated 20 times with the four language and allocator variants interleaved inside every repeat, so that thermal or background drift falls on all of them alike, and every point is a median with a bootstrap 95% confidence interval [20, 24]. The representative operating point is not chosen by eye but taken as the busiest swept load at which all four variants still hold p99 below 5 ms, 56 kops/s against a measured saturation knee of 335 kops/s. All of it runs on a dedicated dual-socket Xeon pinned to cores 2–15, and that host bounds what the campaign may claim, because it is a KVM guest. The frequency governor and the turbo state are unobservable from inside it, so frequency stability is assumed instead of enforced, and CPU steal is non-zero at roughly the order of the latencies under study. Run-level statistics survive that, which is why the overload and the resource comparisons are resolved here. The far tail below saturation does not, so it is reported as unresolved and not as a null, and the bare-metal protocol that would settle it is specified and published.

- Below saturation the four variants sit within $2\,\mu$s of one another at the median, 2.1%, and the paired test resolves a difference of $1.6\,\mu$s in Rust's favour under both allocators. Higher up the distribution the instrument runs out: the widest interval on a single variant's own p99 spans $1480\,\mu$s against $293\,\mu$s of spread between all four, and where the uncertainty about one variant exceeds the gap between them, an ordering would be reporting the scheduler. A second limit is structural and no host would fix it. An end-to-end operation is 0.250% the work the language performs, the rest being a cross-thread handoff charged to the operating system [27], so this sweep cannot bound a language effect in either direction. Measured on its own the bare operation is 1.6× faster in Rust, and

the likely cause is the standard hash map's layout and nothing about safety.

- Past saturation the languages are not equivalent, and the margin is not marginal. At 784 kops/s offered, C++ sustains 116 kops/s where Rust sustains 63, a factor of 1.8, in 20/20 repeats under both allocators and with the Holm-corrected p at the floor of the whole test family. The slower variant is not simply doing less work: it draws 1.9 times the CPU per completed operation, which is the signature of effort spent without completing anything. Isolating the bounded queue and the completion handle on their own does not reproduce that ordering at significance, so the mechanism sits in the wider handoff path. This is the largest and most reproducible language effect in the study, and a comparison that stopped at nominal load would have concluded the opposite.

- Safety is measured by injecting ten defects, five from inside the region Rust claims to govern and five chosen because safe Rust is expected *not* to prevent them [22]. Every one of the first five is a compile error, so it cannot be written at all, where in C++ each is valid code whose defect appears only if a test drives it and no single sanitiser covers the five. All five boundary cases compile in safe Rust and misbehave, and on two of them the C++ tooling is the one that reports: LeakSanitizer flags the reference cycle that Rust leaks in silence, and UndefinedBehaviorSanitizer flags the overflow that Rust defines to wrap. ThreadSanitizer catches the lock-order inversion on a minority of runs only, because it builds its lock-order graph from acquisitions that succeed and a run whose first inversion blocks never closes the cycle. Two Rust model checkers do report both original boundary defects, but only after each was re-imported against the checker's own primitives and shrunk to model size [28], so that half of the boundary is caught at test time and after a rewrite.

- Resource use separates the variants where latency could not, and it holds two effects of unequal size. The Rust build costs 1.9–3.4 MiB more peak resident set than the C++ build under either allocator, while linking `jemalloc` at all costs 32.9–34.4 MiB in either language. Both are real and the allocator's is an order of magnitude the larger. The language difference is a fixed cost and not a per-context one, and no hardware counters were collected, so it is established without being explained. CPU inverts with the regime, marginally lower for Rust while the service copes and much higher per completed operation once it does not. The migration seam is small on every axis measured: 11 C entry points in

231 lines of Rust, all raw-pointer work confined to 9 functions with the core left free of `unsafe`, and no cost detected against the same operation called natively, which bounds any per-operation tax at 10.91%.

8 Discussion

- RQ1's answer divides at saturation, and reporting either half on its own would misrepresent it. Below the knee the two languages are equivalent in central behaviour by a margin no deployment would notice, which is a result about services of this shape, and the far tail in the same regime stays unresolved, which is an admission about the instrument. Past the knee they diverge by 1.8×. The closest prior work measures this quantity for 5G control-plane procedures, finds the same shape, and traces it to stackless `async/await` [8]. That attribution is accepted here. The point is that the shape appeared with no asynchronous runtime on either side and the concurrency model held fixed, so a runtime cannot be the only place such an effect originates. The prediction registered in advance also named where to look first, bounds and overflow checking, and that part of it was wrong. Recording the miss is worth more to the next study than quietly substituting the explanation that survived.

- On safety the difference is one of kind, and the boundary is reported as a first-class result and not as a caveat. A compiler that refuses to build a program rules the defect out of every execution, where a sanitiser rules it out only of the executions an instrumented test happens to take. What is absolute there is the execution coverage, not the breadth of the catalogue. Rust reasons about aliasing and not about acquisition order, so a `Mutex` guard whose lifetime the borrow checker tracks perfectly says nothing about whether two threads take their locks in a compatible order [29]. The lost update escapes ThreadSanitizer for the same reason it escapes the borrow checker: where every access is correctly synchronised, there is no unsynchronised access to find. So adopting Rust retires one category of concurrency defect and leaves untouched the category that fails silently, and a migration plan reading "memory-safe" as "concurrency-safe" would mis-budget its review effort. One case reaches past the ten injected programs and narrows the credit due on both sides. A specification gap let an operation be submitted outside the service's running window, and it survived every test, every parity vector and all three sanitisers because nothing ever called that path, which is the weakness of detection conditional on coverage. What turned undefined behaviour

into a diagnosable failure on the Rust side was an ordinary bounds check and not a proof, so the property worth budgeting for is “checked by default” and not “ruled out at compile time”. The root cause belongs to the method: a prohibition stated without a mechanism behind it leaves each implementation to fail in its own idiom.

- RQ3 splits by resource and the splits do not point the same way, while RQ4 comes out cheap in the machine and expensive in the people. The memory penalty reproduces under both allocators and is credible beyond its p-value, peak resident set being a run-level maximum insensitive to the jitter that destroys a p99, and it is a few MiB of fixed cost against an allocator effect an order of magnitude larger. One close study reports the opposite sign, with Rust lowest and the gap widening as concurrency rises [6]. The two are compatible: their C++ runs on Boost.Fibers, which allocates a stack per fiber, and their Rust on Tokio, where a task is a small heap-allocated future, so what they measured is the per-context cost of two concurrency runtimes. Two further studies agree with the direction found here [9, 30]. On migration the dominant cost is human [13, 31], and confining `unsafe` to the seam bounds what has to be reviewed and not what can be corrupted, since host and component share one address space [32, 33].

- The three hypotheses were committed before any measurement, and stating their disposition is what separates a test from a description. H1 is supported below saturation and refuted above it, and not in the direction its wording anticipated, since the difference appeared in overload and not in the far tail H1 actually named. It was also not well posed, because “performance-equivalent” reads as one quantity and names at least three. H2 holds without exception for the five injected classes of its kind and fails in the looser reading its wording invites, so it is accepted for memory safety and data-race freedom and rejected for concurrency correctness at large. H3 is supported by the bound on a cost the measurement could have missed and not by the failure to detect one. Three threats stay load-bearing behind all of it. One author wrote both implementations, and the parity suite bounds behaviour and not performance [10]. The end-to-end instrument measures the design faithfully and the language barely at all. And the overload divergence is replicated with its mechanism narrowed and not closed.

- The guidelines are the practical form of all this, and each carries the limit of the

finding behind it. Choose a first component for the narrowness and the scalarity of its interface instead of for its size, because the redesign of a rich interface is the schedule risk and the boundary itself is not. Keep the C ABI in its own crate and the core free of `unsafe`, which makes the reviewable surface a countable thing [34]. Leave the concurrency review budget where it is. Prefer a shared-nothing structure in either language, since the strongest concurrency result here belongs to the design and to neither compiler. Plan the learning curve as a training cost and instrument it, so the second component can be scheduled on measured effort. Measure the component as well as the round-trip, and sweep past the point where the service stops coping, which is the guideline this study would most like to have followed from the start.

9 Conclusion and Future Work

- The thesis compared modern C++ and Rust on a UE Context Management service modelled on a real gNB-CU-CP component, along four dimensions: tail-latency predictability, compile-time defect prevention, safety-model overhead, and incremental-adoption cost. Both implementations were built against one frozen specification and driven along identical code paths, which is what lets a language effect be told apart from a difference of design or of skill.

- On safety the difference is one of kind and it has a definite edge, and the edge stops where the catalogue says it does. Safe Rust rejects all five injected memory-safety and data-race classes at compile time, turning hazards that C++ can only hope to catch with instrumented tests into programs that cannot be written. All five boundary classes compile and misbehave in safe Rust as they do in C++, and on two of them the C++ tooling reports what Rust's does not. Stated precisely, Rust eliminates data races and aliasing errors. It does not eliminate race conditions, lock-ordering errors, leaks, wrapping arithmetic or the blast radius of a failed worker.

- On performance the evidence divides twice before it divides by resource, and getting both divisions right was this study's most consequential correction to itself. It divides first by what is being measured, because an end-to-end operation here is 0.250% language and the rest an operating-system handoff, so an end-to-end null bounds nothing about either compiler, while the same operation measured apart is 1.6× faster in Rust. It divides second by whether the service is coping. Below saturation the two are equivalent at the median and their far tail is unresolvable on a virtualised host, and past saturation C++ sustains 1.8× the throughput of Rust while drawing 1.9× the CPU per completed operation. On memory the Rust build carries a few MiB more fixed footprint under both allocators, an order of magnitude less than the allocator choice costs.

- What follows is a narrow claim, narrower than this thesis expected to make. An unmodified Rust core can be driven from C++ across a thin seam whose cost on a real operation is bounded at 10.91%, with all `unsafe` confined to the seam

and the core verified free of it. For stateful, concurrent, partitionable control-plane services of this kind, and at nominal load, the choice between the two languages is not a performance decision. It is a decision about which classes of defect an organisation wants to make unwriteable, and about whether it will pay the engineering and learning cost of getting them. Under overload it becomes a performance decision again, and one that currently favours C++. What the thesis adds is therefore an instrument and not a verdict: a frozen specification, a parity-tested pair of implementations, a measurement framework that reaches past the saturation knee, and a defect catalogue that maps where the guarantee stops.

- Future work answers the limitations one at a time. The immediate priority is the bare-metal campaign, whose target is now narrow: the far tail below saturation, with hardware counters and allocation profiles collected on the same visit, which is what would turn the memory result from an observation into an explanation. Beyond it, replaying anonymised production traces would validate the operation mix, a subsystem with genuinely shared mutable state would exercise the two languages differently than a shared-nothing design does, evaluating Rust async runtimes against the thread-per-shard model would separate the runtime from the language in the tail result [8], and measuring what in-process isolation of the two heaps costs would price what protecting an `unsafe`-free core is worth [32]. A longer case study tracking defect density and review effort during an incremental adoption is what would settle the human cost this study can only name.

Statement on the Use of AI Tools

Tool. Anthropic's Claude Opus 5, used through the Cursor development environment, is the only generative AI system used in the preparation of this thesis. It was used throughout the work, from June to August 2026. Following University of Helsinki policy the model is neither named as an author nor cited as a source.

What the assistance covered. It was extensive, and is declared as such. The model drafted and revised the running prose of this thesis and proposed the order in which its arguments are made. It wrote and refactored the two implementations of the UE Context Management service, the load generator, the fault-injection catalogue, and the interoperability prototype. It wrote the analysis code behind the bootstrap intervals, the exact paired permutation tests, and the Holm correction, together with the figure-generating and build scripts and the LaTeX sources. It also suggested candidate literature. The abstract is the one part of this thesis to which that assistance must not extend, because the University's thesis guide does not permit a language model in preparing the thesis summary, which in this programme is also the maturity test. In this draft it has not yet been rewritten unaided, and this statement will say so until it has been.

What the model did not settle. The research problem, the four research questions, and the three hypotheses are the author's, as is the control on which the study's value rests: freezing one language-neutral specification and one concurrency model, so that the language remains the only systematic difference between the two implementations. So is the decision that keeps RQ2 falsifiable, which was to carry defect classes from outside Rust's compile-time guarantee so that the premise could be embarrassed rather than merely confirmed. Every measurement reported here was run by the author on the hosts documented in this thesis, and no measured value came from a language model. Each measured number reaches the text through a macro regenerated from the raw logs, so a figure cannot be transcribed by hand, by the author or by the model. The readings of those numbers are the author's as well, including the two restraints the evidence imposes: a far tail a virtualised host cannot resolve is not equivalence, and a sweep dominated by cross-thread handoff cannot bound an effect of the language.

How the output was checked. Model output was treated as a draft to be falsified

rather than as a result, and claims were checked against the sources cited for them. Every bibliography entry was resolved against Crossref, DataCite, or the issuing body, and each was traced to the page or clause carrying the claim made of it. That audit found four passages where the prose overstated or misplaced its source, one of them a sentence inside quotation marks that does not appear in the work it was attributed to. All four were corrected, and the audit is kept with the artefacts so that the correction remains reviewable. Code was checked by running it, against one conformance suite whose transcripts are diffed line by line and a fault catalogue that fixes each expected outcome before the defect is injected. Nothing is asserted here on the model's authority.

What the author took from it. The model is dependable at mechanical work, at transposing an idiom between two languages or keeping a build honest. It is undependable in a specific and predictable way: it will supply a plausible citation for a claim and a confident sentence where the data supports a hedge. Its fluency is not evidence. The effort saved in drafting is spent again, and rightly, in checking, and the parts of this thesis that are defensible are the parts that were checked against a source, a specification, or a measurement.

Responsibility. The author is responsible for the whole of this thesis: its language, its facts, its code, its measurements, and its conclusions. Where it is wrong, the error is the author's.

Bibliography

[1] 3GPP. *Study on scenarios and requirements for next generation access technologies*. Technical Report (TR) 38.913. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms. 3rd Generation Partnership Project (3GPP), Apr. 2024. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.913/38913-i00.zip.

[2] L. Szekeres, M. Payer, T. Wei, and D. Song. "SoK: Eternal War in Memory". In: *Proceedings of the 2013 IEEE Symposium on Security and Privacy (S&P)*. 2013, pp. 48–62. DOI: 10.1109/SP.2013.13. URL: https://doi.org/10.1109/SP.2013.13.

[3] S. Klabnik and C. Nichols. *The Rust Programming Language*. 2nd. San Francisco, CA: No Starch Press, 2023. ISBN: 978-1-7185-0310-6.

[4] J. Almodovar and A. Sultan. *The SA1 road to 6G*. 3GPP Highlights, Issue 09. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work; ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026. Dec. 2024. URL: https://www.3gpp.org/news-events/3gpp-news/sa1-6g-road.

[5] Cybersecurity and Infrastructure Security Agency. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously*. Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps.

[6] R. Karlbäck and A. Orö. "Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2021. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307.

[7] J. Olsson and I. Stenström. “Rust Async for Thread-per-Core Packet Processing”. Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE. Master’s thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-197029.

[8] A. Arvidsson and M. Kaller. “Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures”. Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master’s thesis. Linköping University, Department of Computer and Information Science, 2025. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746.

[9] V. Olsson. “Rust Programming Language as an Alternative to C for RAN Management Applications”. Carried out at Ericsson AB. Master’s thesis. KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-325822.

[10] C. Emberg and A. Wadsten. “Attainable Safety and Long-Term Resource-Efficiency Using Rust: Evaluating the Viability of Rust in Terms of Development Effort and Performance for the Decompilation of ASN.1 Messages”. Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE. Master’s thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-195552.

[11] R. Gjomemo, P. H. Phung, E. Ballou, K. S. Namjoshi, V. N. Venkatakrishnan, and L. Zuck. “Leveraging Static Analysis Tools for Improving Usability of Memory Error Sanitization Compilers”. In: *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. Two of the six authors are affiliated with Nokia. 2016, pp. 323–334. DOI: 10.1109/QRS.2016.44. URL: https://doi.org/10.1109/QRS.2016.44.

[12] J. Dean and L. A. Barroso. “The Tail at Scale”. In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794.

[13] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. “Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study”. In: *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton.

[14] 3GPP. *NG-RAN; Architecture description*. Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip.

[15] B. Schroeder, A. Wierman, and M. Harchol-Balter. "Open Versus Closed: A Cautionary Tale". In: *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*. 2006. URL: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale.

[16] G. Tene. *HdrHistogram: A High Dynamic Range Histogram*. Project documentation and reference implementation. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026. URL: https://github.com/HdrHistogram/HdrHistogram.

[17] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "Safe Systems Programming in Rust". In: *Communications of the ACM* 64.4 (2021), pp. 144–152. DOI: 10.1145/3418295. URL: https://doi.org/10.1145/3418295.

[18] 3GPP. *Service requirements for the 5G system; Stage 1*. Technical Specification (TS) 22.261. V18.6.0, Release 18. 3rd Generation Partnership Project (3GPP), Mar. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip.

[19] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. "The Case for Writing Network Drivers in High-Level Programming Languages". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–13. DOI: 10.1109/ANCS.2019.8901892. URL: https://doi.org/10.1109/ANCS.2019.8901892.

[20] A. Georges, D. Buytaert, and L. Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: 10.1145/1297027.1297033. URL: https://doi.org/10.1145/1297027.1297033.

[21] P. Enberg, A. Rao, and S. Tarkoma. “The Impact of Thread-per-Core Architecture on Application Tail Latency”. In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: 10.1109/ANCS.2019.8901874. URL: https://doi.org/10.1109/ANCS.2019.8901874.

[22] R. Natella, D. Cotroneo, and H. S. Madeira. “Assessing Dependability with Software Fault Injection: A Survey”. In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: 10.1145/2841425. URL: https://doi.org/10.1145/2841425.

[23] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. “Detecting Cross-Language Memory Management Issues in Rust”. In: *Computer Security – ESORICS 2022*. 2022, pp. 680–700. DOI: 10.1007/978-3-031-17143-7_33. URL: https://doi.org/10.1007/978-3-031-17143-7_33.

[24] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993. ISBN: 978-0-412-04231-7. DOI: 10.1007/978-1-4899-4541-9. URL: https://doi.org/10.1007/978-1-4899-4541-9.

[25] S. Holm. “A Simple Sequentially Rejective Multiple Test Procedure”. In: *Scandinavian Journal of Statistics* 6 (1979), pp. 65–70. URL: https://www.jstor.org/stable/4615733.

[26] T. Yin. *Lizard: An extensible cyclomatic complexity analyzer for many programming languages*. Version 1.23.0. Accessed August 2026. 2026. URL: https://github.com/terryyin/lizard.

[27] X. Zhou, V. Leis, X. Yu, and M. Stonebraker. “OLTP Through the Looking Glass 16 Years Later: Communication Is the New Bottleneck”. In: *15th Annual Conference on Innovative Data Systems Research (CIDR 2025)*. 2025. URL: https://www.vldb.org/cidrdb/papers/2025/p17-zhou.pdf.

[28] J. Bornholt, R. Joshi, V. Astrauskas, B. Cully, B. Kragl, S. Markle, K. Sauri, D. Schleit, G. Slatton, S. Tasiran, J. Van Geffen, and A. Warfield. “Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3”. In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 836–850. DOI: 10.1145/3477132.3483540. URL: https://doi.org/10.1145/3477132.3483540.

[29] Y. Zhang, K. Zhang, G. Liu, Y. Cai, and S. Qin. "Two Birds One Stone: Effective Static Detection of Resource and Communication Deadlocks in Rust Programs". In: *Automated Software Engineering* 33.2, 65 (2026). Extends the preprint arXiv:2401.01114. DOI: 10.1007/s10515-026-00614-z. URL: https://doi.org/10.1007/s10515-026-00614-z.

[30] A. Sundvisson and S. Hofer. "User-Space Device Drivers on Embedded Linux: A Comparative Case Study of C and Rust". Bachelor's thesis. Blekinge Institute of Technology, Department of Software Engineering, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-29775.

[31] W. Crichton and S. Krishnamurthi. "Profiling Programming Language Learning". In: *Proceedings of the ACM on Programming Languages* 8.OOPSLA1 (2024), pp. 29–54. DOI: 10.1145/3649812. URL: https://doi.org/10.1145/3649812.

[32] E. Rivera, S. Mergendahl, H. Shrobe, H. Okhravi, and N. Burow. "Keeping Safe Rust Safe with Galeed". In: *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*. 2021, pp. 824–836. DOI: 10.1145/3485832.3485903. URL: https://doi.org/10.1145/3485832.3485903.

[33] P. Kirth, M. Dickerson, S. Crane, P. Larsen, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, and M. Franz. "PKRU-Safe: Automatically Locking Down the Heap Between Safe and Unsafe Languages". In: *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*. 2022, pp. 132–148. DOI: 10.1145/3492321.3519582. URL: https://doi.org/10.1145/3492321.3519582.

[34] A. N. Evans, B. Campbell, and M. L. Soffa. "Is Rust Used Safely by Software Developers?" In: *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 2020, pp. 246–257. DOI: 10.1145/3377811.3380413. URL: https://doi.org/10.1145/3377811.3380413.